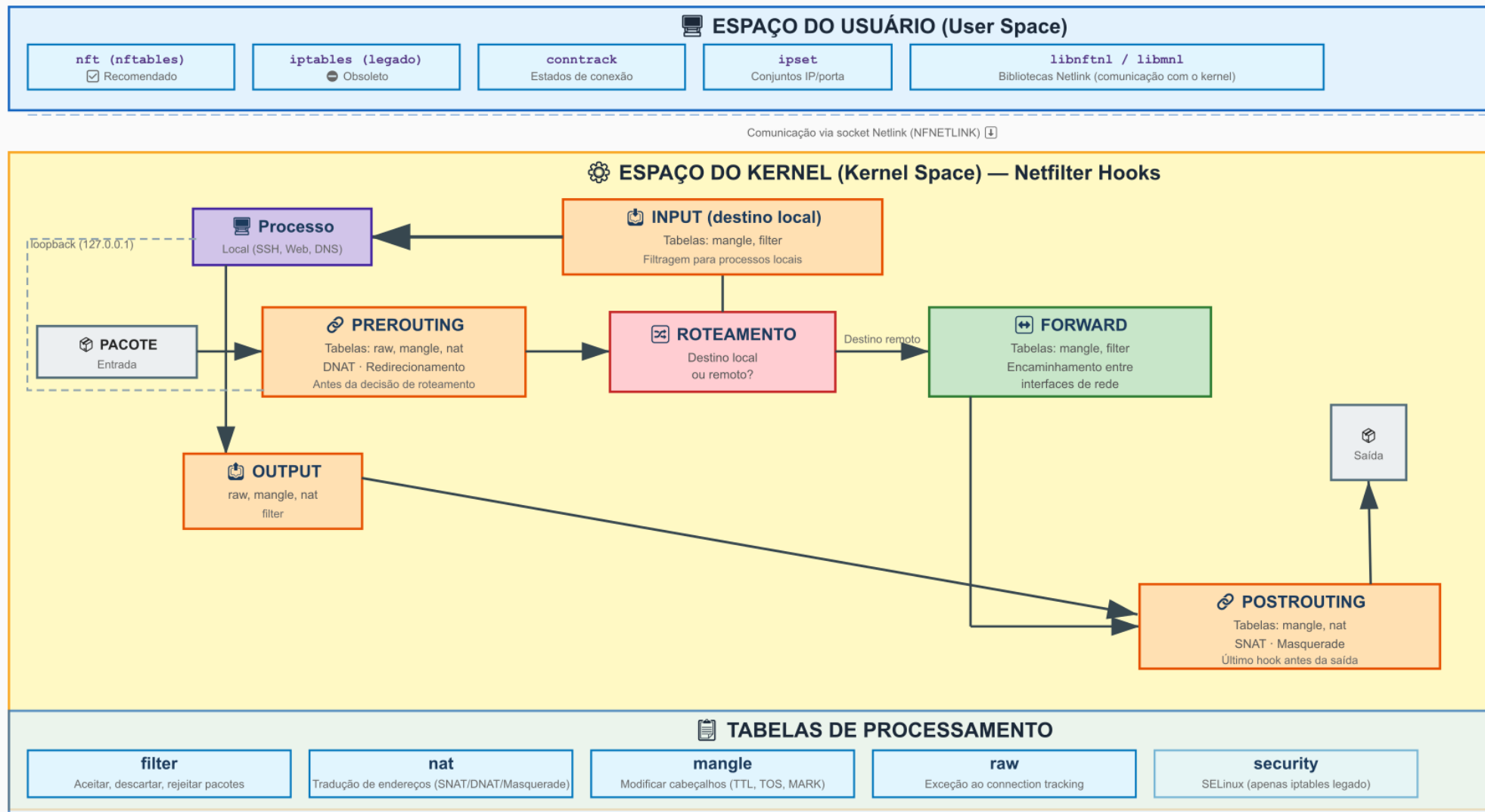


**Segurança em Sistemas Operacionais e Redes de
Computadores I**
ISG012-06 - Netfilter - Iptables - Nftables - Teoria

Objetivo

- Compreender a arquitetura interna do framework Netfilter do Linux, dominando o funcionamento das tabelas, chains e regras de filtragem.

Arquitetura Conceitual do Netfilter Framework



Objetivo

- Compreender a arquitetura do Netfilter e seu posicionamento no kernel Linux
- Diferenciar as ferramentas iptables (legado) e nftables (moderna)
- Dominar o conceito de tabelas (filter, nat, mangle) e chains (INPUT, FORWARD, OUTPUT, PREROUTING, POSTROUTING)
- Aplicar regras de filtragem stateful versus stateless
- Configurar NAT básico (SNAT e DNAT) para compartilhamento de conexão
- Implementar logging efetivo de tentativas de intrusão

Introdução ao Netfilter

- O Netfilter representa o framework de filtragem de pacotes integrado ao kernel Linux desde a versão 2.4, funcionando como a infraestrutura fundamental para firewall, NAT e manipulação de pacotes;
- Diferente de soluções externas, o Netfilter opera diretamente no espaço do kernel, garantindo alta performance e baixa latência no processamento de regras;
- Sua arquitetura modular permite interceptar pacotes em pontos específicos do stack de rede, possibilitando decisões de segurança baseadas em endereços, portas, protocolos e estado da conexão, constituindo a base para ferramentas como iptables e nftables.

Introdução ao Netfilter

- Framework oficial do kernel Linux para processamento de pacotes
- Opera no kernel space (nível de privilégio mais alto)
- Suporta filtragem stateful (connection tracking)
- Permite manipulação de pacotes via hooks em 5 pontos críticos
- Base para implementação de firewalls, NAT e QoS
- Substituído o ipchains (Linux 2.2) por ser mais flexível e escalável

Evolução: iptables vs nftables

- O iptables dominou a administração de firewalls Linux por mais de duas décadas, utilizando estruturas de regras separadas para IPv4 e IPv6, com sintaxe complexa e baixa performance em regras extensas;
- O nftables, introduzido no kernel 3.13 e consolidado nas distribuições modernas como o Linux Mint 21+, representa uma revolução arquitetural;
- Utilizando uma máquina virtual de bytecode no kernel, o nftables unifica o tratamento de IPv4/IPv6, oferece sintaxe mais legível, suporta estruturas de dados avançadas (sets e maps) e permite atualizações atômicas de regras sem interromper o tráfego existente.

Evolução: iptables vs nftables

- iptables (Legado):
 - Sintaxe complexa e verbosa
 - Tabelas separadas para IPv4 (iptables) e IPv6 (ip6tables)
 - Performance degradada com milhares de regras
 - Dificuldade em debugar regras complexas
- nftables (Moderno/Recomendado):
 - Sintaxe unificada e intuitiva (família inet cobre IPv4/IPv6)
 - Compilação para bytecode eficiente
 - Suporte nativo a sets, maps e concatenations
 - Atualizações atômicas via nft command
 - Default no Linux Mint 20+ e Ubuntu 20.04+

Arquitetura de Tabelas e Chains no nftables

- A No nftables, a estrutura de controle de pacotes é organizada de forma flexível e hierárquica, diferente do modelo rígido do iptables legado;
- O administrador cria tabelas com nomes arbitrários e define sua família de endereçamento — inet para IPv4 e IPv6 simultaneamente, ip para IPv4 exclusivo, ou ip6 para IPv6 exclusivo;
- Dentro de cada tabela, são criadas chains que especificam o tipo de processamento (filter para segurança, nat para tradução de endereços, route para roteamento especializado), o hook (ponto de interceptação no fluxo do pacote como input, forward, output, prerouting ou postrouting), a prioridade de execução e a política padrão (accept ou drop);
- Essa arquitetura permite que uma única tabela contenha múltiplas chains com funções distintas — por exemplo, uma chain de filtragem e outra de NAT coexistindo na mesma tabela inet firewall;
- O processamento ocorre sequencialmente dentro de cada chain até que uma ação terminal (accept, drop, reject) seja executada, determinando o destino do pacote;
- Essa flexibilidade elimina a necessidade de tabelas pré-definidas como raw ou security existentes no iptables, simplificando a administração e reduzindo erros de configuração.

Arquitetura de Tabelas e Chains no nftables

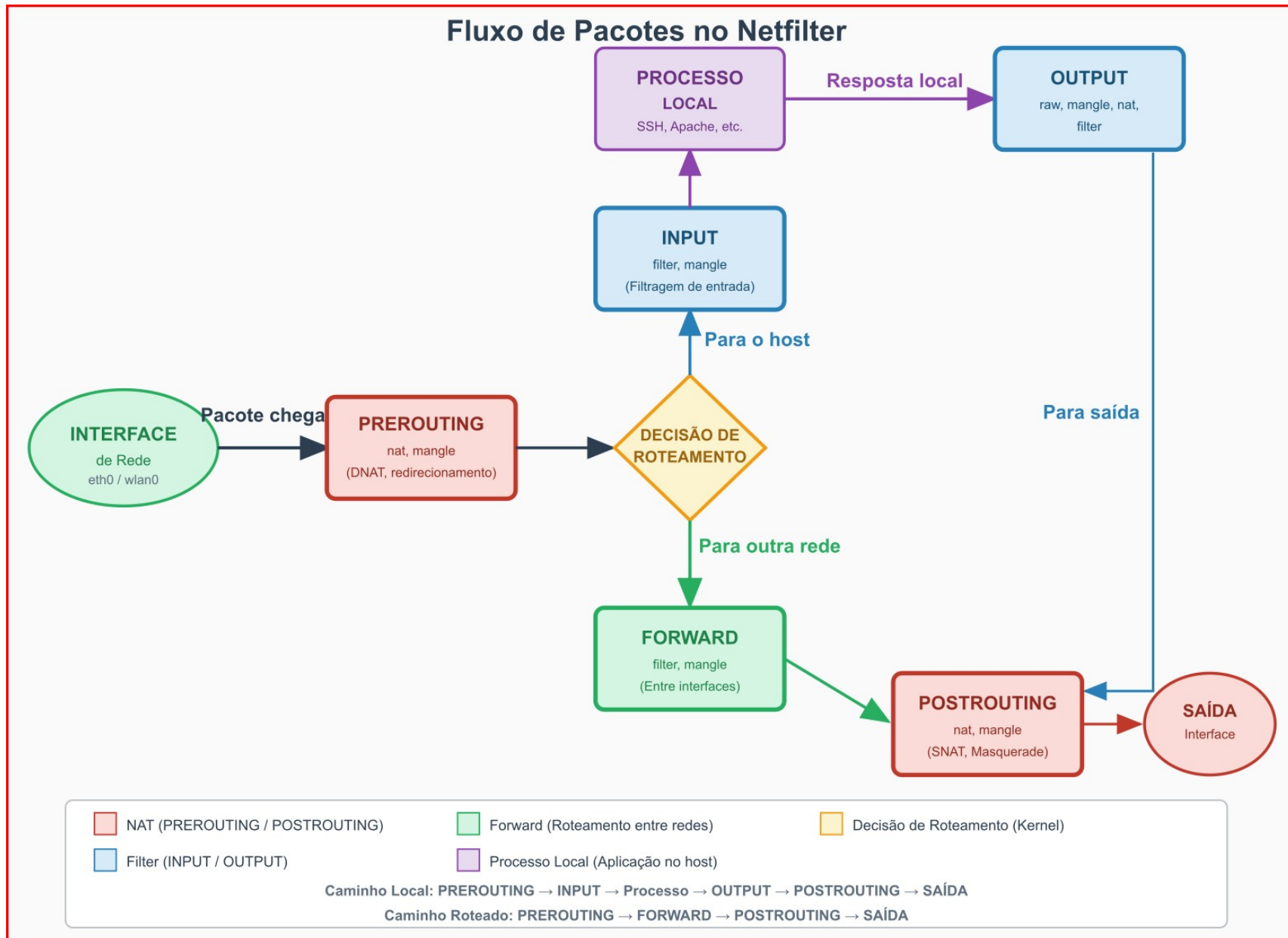
- Conceito geral do Netfilter (válido para ambos):
 - filter: Decisão de aceitar ou descartar pacotes (segurança padrão)
 - nat: Tradução de endereços (SNAT, DNAT, masquerade)
 - mangle: Modificação de campos do cabeçalho IP (TTL, TOS, MARK) para QoS
- Como funciona no nftables (moderno):
 - O nome da tabela é definido pelo administrador (ex: filter, nat ou qualquer outro nome)
 - Dentro da tabela, cria-se chains que especificam:
 - Tipo: filter, nat ou route
 - Hook: ponto de interceptação (input, output, forward, prerouting, postrouting, ingress)
 - Prioridade: ordem de processamento no hook
 - Política padrão: accept ou drop
 - Uma mesma tabela pode conter chains de tipos diferentes (ex: uma chain filter e outra nat dentro da mesma tabela inet firewall)
 - Não existem tabelas raw ou security pré-definidas como no iptables legado
- Ações disponíveis nas chains:
 - accept, drop, reject (com mensagem ICMP), log, counter, masquerade, dn timer, snat

Chains e Fluxo de Pacotes

- As chains representam os pontos de interceptação no caminho do pacote através do kernel;
- Quando um pacote chega à interface de rede, atravessa primeiro a chain PREROUTING (onde ocorre o DNAT), seguindo para a decisão de roteamento;
- Se destinado ao próprio host, direciona-se à chain INPUT; se destinado a outra rede, percorre a chain FORWARD e posteriormente POSTROUTING (onde ocorre o SNAT). Pacotes gerados localmente iniciam pela chain OUTPUT e também passam por POSTROUTING;
- Essa arquitetura de ganchos permite visibilidade total do tráfego em múltiplos estágios.

Chains e Fluxo de Pacotes

- PREROUTING: Primeiro ponto de contato (DNAT, redirecionamento de portas)
- INPUT: Pacotes destinados ao próprio host (serviços locais)
- FORWARD: Pacotes roteados entre interfaces (gateway/roteador)
- OUTPUT: Pacotes gerados por processos locais
- POSTROUTING: Último ponto antes da transmissão (SNAT, mascaramento)
- Decisão de roteamento: Determina se pacote é local ou encaminhado



Sintaxe do nftables

- O nftables utiliza uma sintaxe declarativa e hierárquica através do comando nft, organizando as regras em tabelas, chains e regras propriamente ditas;
- Uma tabela é definida com sua família (ip, ip6, inet para ambos) e nome. Dentro da tabela, as chains são criadas com tipo (filter, nat), gancho de interface (input, forward, output) e prioridade;
- As regras utilizam expressões concisas para matching (ip saddr, tcp dport, ct state) e ações (accept, drop, reject, log, counter), suportando operadores lógicos e agrupamentos complexos através de sets para múltiplos endereços ou portas simultaneamente.

Sintaxe do nftables

- Estrutura básica:
- bash

```
nft add table inet filter
nft add chain inet filter input { type filter hook input priority 0 \; policy drop \; }
nft add rule inet filter input tcp dport 22 accept
```
- Componentes:
 - Família: inet (IPv4+IPv6), ip, ip6, arp, bridge
 - Tipo: filter, nat, route
 - Gancho (hook): input, output, forward, prerouting, postrouting, ingress
 - Prioridade: Ordem de processamento no gancho (0 = padrão)
 - Política padrão: accept (recomendado para evitar lockout) ou drop

Stateful vs Stateless Inspection

- A inspeção stateful representa a capacidade do firewall de monitorar o estado das conexões através do módulo conntrack (connection tracking), diferenciando pacotes que iniciam novas conexões daqueles pertencentes a sessões estabelecidas;
- Essa distinção permite criar regras altamente seguras onde apenas respostas a solicitações legítimas são aceitas, bloqueando implicitamente pacotes malformados ou ataques de scanning;
- Em contraste, firewalls stateless analisam cada pacote isoladamente, sem contexto de conexão, exigindo regras explícitas bidirecionais e sendo vulneráveis a spoofing de sessões, embora ofereçam menor overhead computacional.

Stateful vs Stateless Inspection

- Stateful (Recomendado):
 - Mantém tabela de conexões ativas (/proc/net/nf_conntrack)
 - Estados: new, established, related, invalid, untracked
 - Regra eficiente: ct state established,related accept
 - Proteção contra pacotes fora de sequência (TCP sequence checking)
 - Suporte a protocolos complexos (FTP, SIP) via helpers
- Stateless:
 - Análise individual de cada pacote
 - Necessita regras explícitas para tráfego de retorno
 - Menor consumo de memória (ideal para dispositivos embarcados)
 - Vulnerável a ataques de fragmentação e spoofing

Connection Tracking

- O connection tracking (conntrack) constitui o mecanismo central da inspeção stateful no Linux, armazenando informações sobre todas as conexões de rede em uma tabela hash no kernel. Para cada fluxo, mantém metadados incluindo protocolo, endereços de origem/destino, portas, estado atual (SENT, RECEIVED, ESTABLISHED) e timeout;
- O sistema utiliza protocol trackers específicos para inspecionar camadas de aplicação, permitindo que protocolos como FTP (modo passivo), SIP e H.323 funcionem atrás de NAT;
- A visualização da tabela de conexões é realizada via comando conntrack -L, sendo essencial para troubleshooting de regras de firewall.

Connection Tracking

- Tabela de conexões: Armazenada em `/proc/net/nf_conntrack`
- Estados principais:
 - NEW: Primeiro pacote da conexão (SYN sem resposta)
 - ESTABLISHED: Handshake TCP completo ou tráfego UDP bidirecional
 - RELATED: Nova conexão relacionada a uma existente (ex: FTP data channel)
 - INVALID: Pacote que não pertence a nenhuma conexão conhecida (descartar!)
- Timeouts: TCP estabelecido (5 dias), UDP (30 segundos), SYN-SENT (2 minutos)
- Limitações: Consumo de memória proporcional ao número de conexões

Configuração de Regras Básicas

- A configuração efetiva de um firewall inicia-se com a definição da política padrão restritiva, recomendando-se iniciar com policy accept durante a configuração remota para evitar bloqueio acidental (lockout), migrando para policy drop após validação;
- As regras essenciais incluem permitir tráfego de loopback (lo interface), aceitar conexões estabelecidas, liberar acesso SSH administrativo (tcp/22) e habilitar consultas DNS (udp/53);
- A sintaxe do nftables permite combinações complexas usando sets para múltiplas portas e endereços, além de suportar logging integrado para auditoria de tentativas de acesso bloqueadas.

Configuração de Regras Básicas

- Regras essenciais de segurança:
 1. Loopback: `iif lo accept` (interface interna sempre liberada)
 2. Estado: `ct state established,related accept` (stateful)
 3. SSH: `tcp dport 22 ct state new accept` (administração remota)
 4. DNS: `udp dport 53 accept` (resolução de nomes)
 5. ICMP: `icmp type echo-request accept` (ping para diagnóstico)
- Boas práticas:
 - Utilizar counter para monitorar hit counts de regras
 - Adicionar comentários (`comment "Descrição"`) para documentação
 - Testar com `nft -f /etc/nftables.conf` antes de aplicar permanentemente

NAT - Network Address Translation

- O NAT resolve a escassez de endereços IPv4 permitindo que múltiplos dispositivos em rede privada compartilhem um único endereço IP público;
- O SNAT (Source NAT) modifica o endereço de origem quando pacotes saem da rede interna para a internet, tipicamente usando mascaramento dinâmico (masquerade) para interfaces com IP variável (DHCP);
- O DNAT (Destination NAT) redireciona conexões externas para servidores internos, como publicar um servidor web da LAN para a WAN através do redirecionamento de portas;
- Essas traduções ocorrem nas chains POSTROUTING (SNAT) e PREROUTING (DNAT), respectivamente, sendo fundamentais para segurança de perímetro e conservação de endereços.

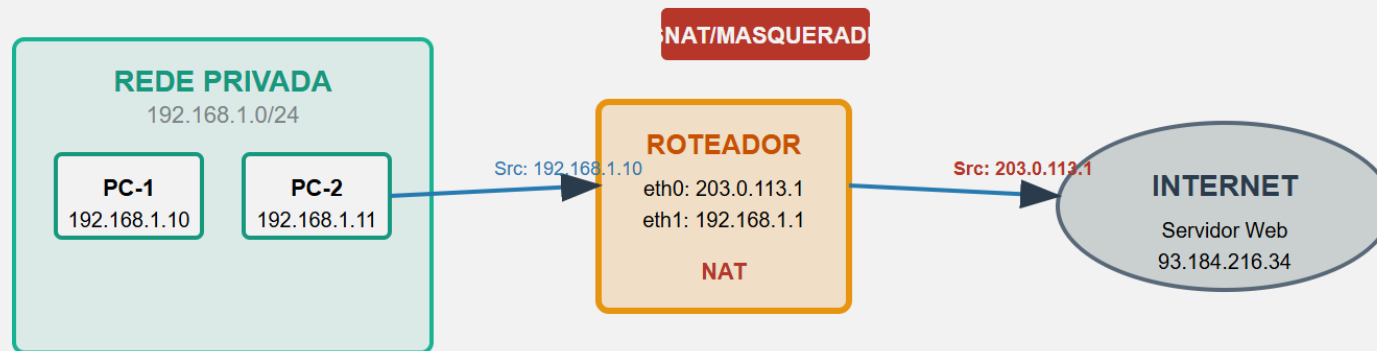
NAT - Network Address Translation

- SNAT (Source NAT / Masquerade):
 - Uso: Rede interna → Internet
 - Chain: POSTROUTING
 - Sintaxe: `oif eth0 masquerade` (IP dinâmico) ou `snat to 203.0.113.1` (IP fixo)
 - Função: Oculta IPs internos, permite múltiplos hosts com um IP público
- DNAT (Destination NAT / Port Forwarding):
 - Uso: Internet → Servidor interno
 - Chain: PREROUTING
 - Sintaxe: `dnat to 192.168.1.100:80` (redireciona porta 80 para servidor web interno)
 - Função: Publicação de serviços, balanceamento de carga básico

Network Address Translation (NAT)

SNAT (Source NAT / Masquerade)

Rede Privada → Internet (Múltiplos hosts, um IP público)



DNAT (Destination NAT / Port Forwarding)

Internet → Rede Privada (Publicação de serviços internos)



Legenda das Operações:

- SNAT (Postrouting): Altera endereço de ORIGEM (IP privado → IP público)
- DNAT (Prerouting): Altera endereço de DESTINO (IP público → IP privado específico)

Logging e Monitoramento

- O logging eficiente em firewalls Linux é fundamental para detecção de incidentes e análise forense;
- O nftables integra a ação log diretamente nas regras, permitindo registrar tentativas de conexão bloqueadas ou suspeitas no journal do sistema (systemd-journald) ou syslog;
- Configurações avançadas permitem especificar o nível de prioridade (warn, info), prefixos identificáveis para facilitar a filtragem de logs, e limitação de taxa (rate limit) para prevenir flooding do disco em caso de ataques distribuídos;
- A análise desses logs revela padrões de port scanning, tentativas de brute force e varreduras automatizadas de vulnerabilidades.

Logging e Monitoramento

- Logging e Monitoramento (Bullet Points)
- Configuração de logging:
bash
nft add rule inet filter input tcp dport 22 ct state new limit rate 3/second log prefix "SSH_ATTEMPT: " accept
- Parâmetros importantes:
 - prefix: Identificador para grep/filtragem (ex: "FIREWALL_DROP:")
 - level: Prioridade do log (emerg, alert, crit, err, warn, notice, info, debug)
 - limit rate: Previne log flooding (ex: limit rate 10/minute)
 - group: Netlink group para logs em userspace (0 = kernel log)
- Análise de logs:
 - journalctl -k | grep FIREWALL (visualização em tempo real)
 - tail -f /var/log/kern.log (syslog tradicional)
 - Scripts Python para análise estatística de origens bloqueadas

Boas Práticas de Segurança

- A implementação segura de firewalls exige adesão a princípios fundamentais: inicie sempre com política padrão permissiva durante a configuração inicial para evitar bloqueio remoto irreversível, migrando para política restritiva apenas após validação completa das regras;
- Mantenha regras específicas antes de regras genéricas, pois o nftables processa sequencialmente e para na primeira correspondência;
- Utilize whitelist (permitir apenas o necessário) em vez de blacklist (bloquear o malicioso). Implemente rate limiting em serviços expostos como SSH para mitigar brute force;
- Documente todas as regras com comentários descritivos e mantenha backups versionados das configurações em `/etc/nftables.conf`.

Boas Práticas de Segurança

- Política default: Iniciar com policy accept, migrar para drop após testes
- Ordem das regras: Específicas primeiro, genéricas por último (primeiro match wins)
- Princípio do menor privilégio: Liberar apenas portas/protocolos estritamente necessários
- Rate limiting: limit rate over 3/second para prevenir DoS em serviços
- Backup: `nft list ruleset > /backup/firewall-$(date +%Y%m%d).conf`
- Testes: Sempre validar em ambiente controlado antes de produção
- IPv6: Não esquecer a família inet (IPv4+IPv6) ou configurar ip6tables separadamente

Checklist de Validação

- Antes de considerar a configuração do firewall como concluída, execute um checklist sistemático de validação:
 - Verifique se o loopback (lo) está explicitamente permitido para evitar quebra de funcionalidades locais;
 - Confirme que conexões estabelecidas são aceitas para garantir o retorno do tráfego legítimo;
 - Teste cada regra individualmente usando o comando counter para verificar se está sendo atingida; valide o acesso aos serviços críticos (SSH, DNS) a partir de hosts autorizados;
 - Verifique se o logging está capturando eventos relevantes sem gerar ruído excessivo;
- Simule cenários de failover (refere-se a mecanismos de contingência que impedem o bloqueio permanente do administrador (lockout) caso uma regra de firewall seja aplicada incorretamente) para garantir que a política default não bloqueie acesso administrativo de emergência.

Checklist de Validação

- [] Loopback interface (lo) configurado para accept
- [] Stateful inspection habilitado (established, related)
- [] Regras de gerenciamento (SSH) testadas e funcionando
- [] DNS (UDP/53) e NTP (UDP/123) liberados se necessário
- [] ICMP básico permitido (echo-request para diagnóstico)
- [] Counters verificados em regras críticas (hit counts)
- [] Logs gerando output em /var/log/kern.log ou journal
- [] Backup da configuração salvo em local seguro
- [] Política default alterada para drop apenas após validação completa